Computer Games Development CW208
GDD
Year IV

Jakub Stepien
C00284361

Jakub Stepien
C00284361

13/04/2026

Contents

Acknowledgements

I would like to thank my project supervisor Omer Ali for the positive feedback and helpful ideas on how to improve the game throughout the course of this project. I would also like to thank the other supervisors, Ben O'Shaughnessy, Libor Zachoval, Amr Abdelhafez and Noel O'Hara for all the constructive criticism that I took and improved upon, and to Martin Harrigan for organising and manage this entire project.

Game Overview

Backtrack is a game where the player finds themselves as a cat in shining armour whose goal is to go home after completing their quest in the wilderness. It is a platformer in which the player will navigate through a series of levels with increasing difficulty but will also slightly change the mechanics as they progress. The focus of the game is to learn the new mechanics as they are given to the player. The completion time and death count of the player is also shown at the end of the game so the player can try to beat the game as fast as possible and with the least number of deaths as possible.

Feature Set

General Features

- The world is made up of a series of smaller levels that link seamlessly together. Each level continues from the last.
- It is a 2D side-scroller based game with cute 8-bit/pixelated graphics.
- The player's health system adds some challenge to the game, while also giving the player a chance to fail without instantly being punished.
- An inventory system to show the progression of the player and explain what is added by the items.

Level Editor/Creator

The game has a level creator which is easy to use with automatic tiling for the environment the player navigates. Simply place the tiles as you want and the surrounding tiles will match accordingly. Tiles that can be placed include grass tiles (1), brick tiles (2) and spikes (3).

Gameplay

The movement of the player is key to this game. Learning how fast the character is and how high they can jump is crucial to progressing. Also, the player needs to be able to adapt to changes in the levels, since they pick up some permanent upgrades, they will need to be able to use them to continue through the levels.

The health system is a nice way of giving the player another chance after failing without punishing them instantly. If the player loses health by hitting a spike tile they can step back and try to approach it differently.

The inventory system is useful to explain the different upgrades the player gains, while also reminding them of their movement abilities. For example, the player picks up a double jump potion early on in the game and later they find themselves stuck on how to progress a certain part, they can check their items to see what they can do and if they can somehow combine the mechanics to achieve their goal.

Having the completion time and death count show at the end makes player want to try and beat their time or beat the game with less deaths than their previous attempt.

Controls

Gameplay:

- A/D = Move left/move right respectively
- W = Jump
- Tab = Inventory
- Left Shift = Dash
- Escape = Main Menu

Level Editor:

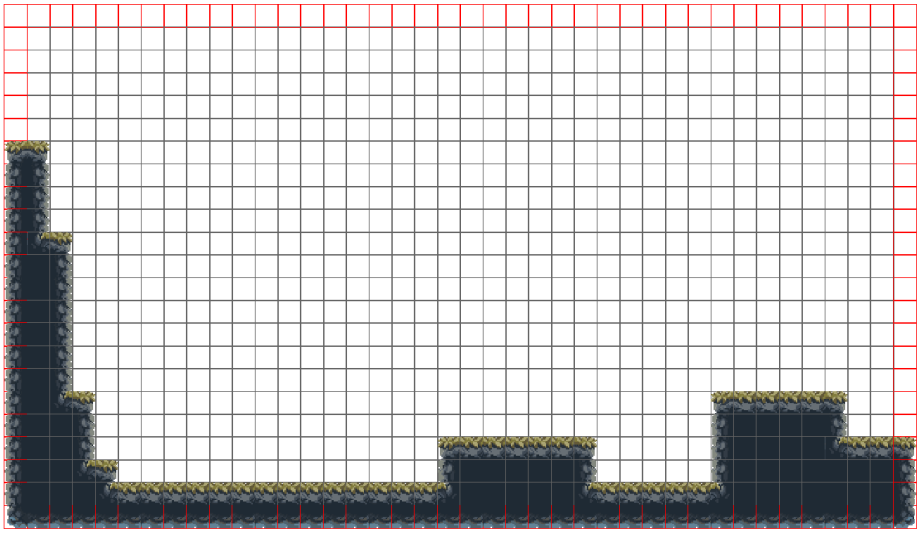
- 1 = Grass tiles
- 2 = Brick Tiles
- 3 = Spikes

The Game World

Overview

Terrain

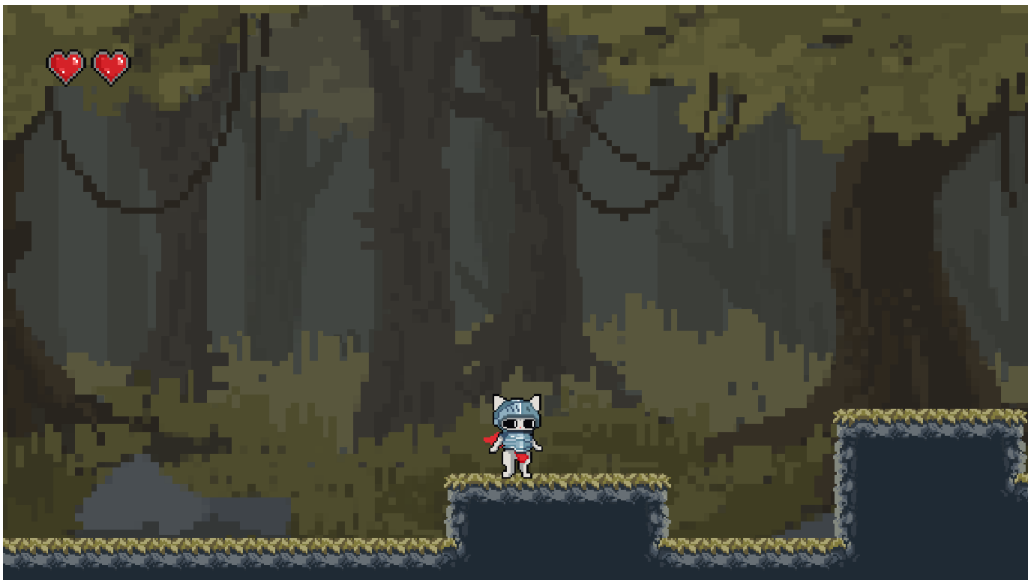
The terrain is made up of a grid of tiles. Each level is a 40x23 grid with 2 rows and columns being off screen to help match the tiles accurately for auto-tiling.



Level example from the level editor, tiles with a red outline are not visible on screen

Background

The background during gameplay is made up of 4 separated sprites and uses a parallax effect to add depth.



Example of background seen during gameplay

The Physical World

Overview

The game is set deep in a forest which the main character is navigating through to get back home from their quest. This is why we can see a forest as the background and most of the terrain consist of grass tiles, the brick tiles being used for a dungeon-like structure later in the game.

Key Locations

Every 4 levels there is an item for the player to pick up on a little pedestal made of grass tiles. It is clear for the player they must go to pick up the item as the cannot progress without it.

Another key location is the dungeon-like structure towards the last few levels where the player must combine all their knowledge from the previous levels to get to the end.

Travel

The player runs, jumps and dashes from platform to platform to cross the terrain.

Scale

Each level is 40x23 tiles equalling to 920 tiles per level, with each tile being 18x18 scaled up two times their size, meaning the level are 1440x828 pixels in size. However, the view is zoomed in slightly on the player during gameplay.

Objects

The main objects the player will come across are items and spikes. When the player collides with spikes, they lose 1 heart and all hearts are lost they are sent back to the start of the level. The items that can be found are potions, each giving the player a new permanent upgrade. There is a potion to increase the player's speed, one to give the player the ability to double jump and one to give the player the ability to dash.

Time

The game is relatively fast paced for a player with experience in platformer games, while a less experienced player might take slightly longer and more attempts to beat some levels. The timer for completion starts when the player starts the game and only stops when they complete the last level.

Rendering System

Overview

Backtrack uses SFML 3.0 as its 2D rendering library, with 3 different views being used depending on the game state, playerView is used during gameplay or when the player is checking their inventory, defaultView for the menu/title screen and levelEditorView for the level editor.

2D/3D Rendering

The game is rendered in 2D using sf::Textures and sf::Sprites from the side perspective.

Camera

Overview

The views are how the camera is controlled, with the only significant difference being the playerView.

defaultView

The camera is at a fixed position to see the entire menu/title screen.

playerView

The camera is slightly zoomed in and follows the player with a slight delay. This gives a nice smooth feel for the player as opposed to a camera that snaps exactly to where the player is.

LevelEditorView

The camera in the level editor is slightly zoomed out to be able to see all of the tiles, including the ones off screen during gameplay because those are used to make the tiles match and look better around the edges.

Game Engine

Overview

Backtrack is made entirely using C++ and the SFML 3.0 library. It runs on a game update loop that is called 60 times a second. Within the main game loop there are switch statements that decide which update and render functions to call based on the current game state.

Game states

The game consists of 6 game states, those being:

- Title screen
- Menu
- LevelEditor
- Gameplay
- Pause
- End

The transitions between the states are done based on the action the player does in the current state. For example, pressing any key takes the player from the title screen to the menu, then pressing the play button will bring the player to the gameplay state.

Collision Detection

I used AABB collision detection using SFML's built in findIntersection() functions. This gets the overlap of any 2 sf::RectangleShapes and from there I use a CollisionResult struct to find out which side of the tile the player is colliding with. This is necessary as I need to be able to check if the player lands on top of a tile to be able to move the player while also only stopping the player velocity if hitting a tile from the bottom or the sides.

To make this slightly more efficient as opposed to checking for a collision between the player and every single tile, I first check if the tile is within 80 pixels of the player, this only checks within a radius of about 2 or 3 tiles around the player, depending on their position.

The World Layout

Overview

The different levels all match up almost seamlessly to give the effect of a large world for the player. When the player completes a level, they are set to the beginning of the next level at the same y-level as the previous level ended to show continuation.

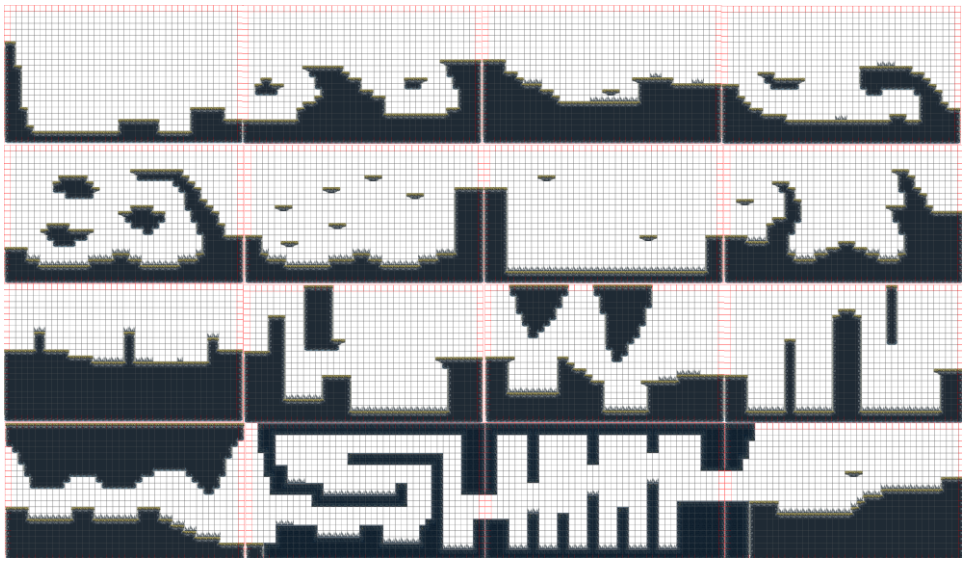


Image of all 16 levels from left-right, top-bottom

Grass layout

Some of the grass level that we can see in the above image give the appearance of a more "natural" look with less straight lines to mimic terrain.

Brick layout

Towards the bottom of the image, we can see more rectangular layouts for levels to mimic the feel of a dungeon-like structure where it looks more like a hand-crafted area.

Game Characters

Overview

The character seen in Backtrack is a cat in shining armour who is just after completing their quest and is now on a voyage back home.

User Interface

Overview

There a few different ways the user interface can be seen in Backtrack. This includes the menus, the player's HUD, the inventory, the level editor and the end screen.

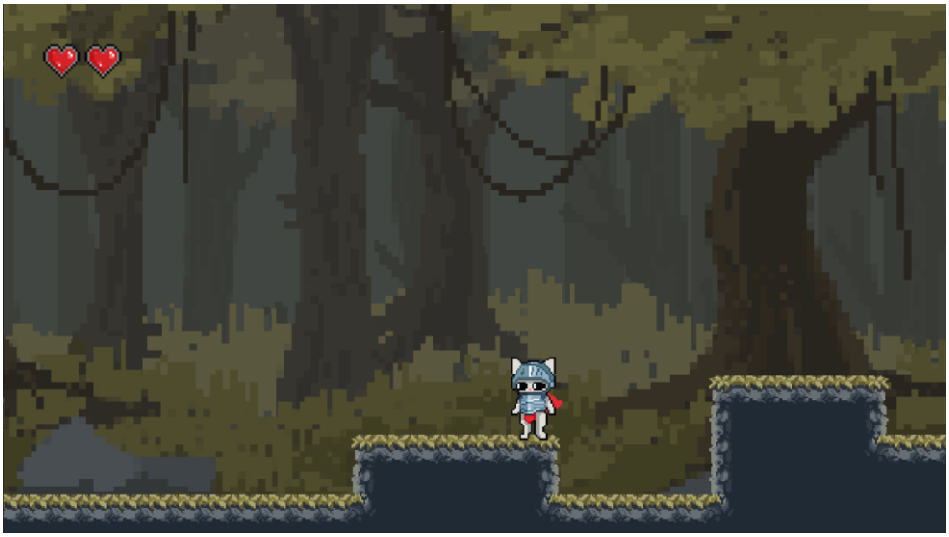
Menu

The menu is simple with a few buttons "Play", "Level Editor", "Controls" and "Exit". Hovering over a button highlights it and when the player presses the left mouse button, it will do that action accordingly.

The menu screen with the play button highlighted

HUD

The HUD is very simple, it only consists of hearts to represent the player's current health in the top-left corner of the screen, if any hearts are empty, that means the player is missing that much health.



Gameplay showing the player's health in the top-left corner

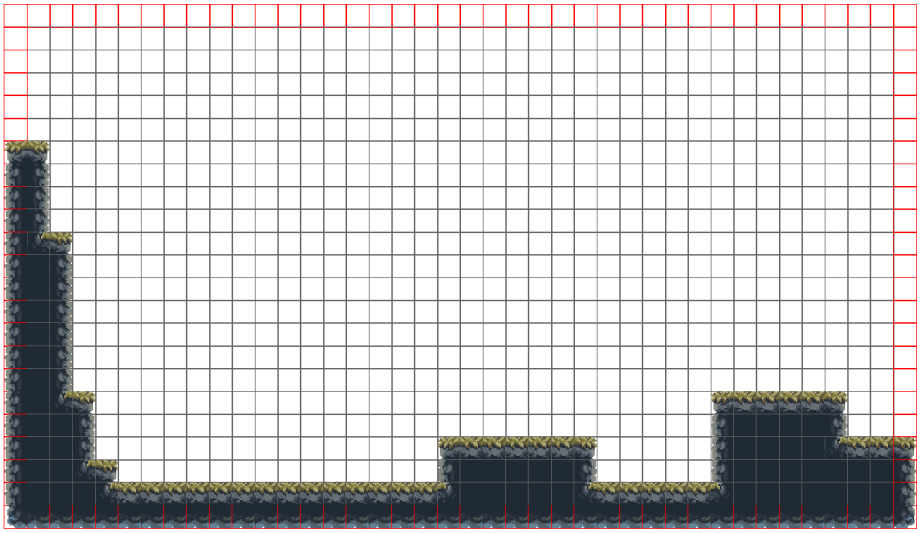
Inventory

Now the inventory UI is a bit more complex. It shows the player's health, along with any items they have picked up. The items can be hovered over with the mouse, this highlights the item and shows what the name of the item is and the item's description in a box above.

Inventory with a potion of speed highlighted and text showing the info box

Level Editor

The level editor shows the grid of tiles with the tiles outlined to be able to accurately place down tiles however the player wants



Level editor showing clickable tiles

End Screen

The end screen shows how fast the player completed the game and how many times they died overall.

End screen after completing the game

Single-Player Game

Overview

Backtrack is a singleplayer game. The gameplay should feel smooth and satisfying to the player, and progressing through the levels should feel rewarding

Here is a breakdown of the key components of the single player game.

Movement

The movement should feel smooth and satisfying making it fun to learn for the player.

Inventory

Seeing the items the player collects should feel rewarding and help with any confusion.

Story

The idea for the story is that the main character has just completed a quest out in the wilderness which is why we can see the cat wearing a suit of armour. The platforming shows the character's venture home after a difficult battle.

Gameplay Time

The game should last somewhere between 5-15 minutes depending on the player's experience with platforming games.

Victory Conditions

The player wins the game after completing all 16 levels.

Character Rendering

Overview

The player is rendered on the window using a sf::Sprite. All the animations and transitions are handled with the functions animate() and setFrames(). The player has a vector of sf::IntRects for the frames of the current player state.

Player animations

The player is constantly playing an animation depending on the current state of the player. If it is a looping animation it uses the animate() function that constantly loops through the vector of sf::IntRects m_playerFrames. While any animation that needs to only be played once uses the playAnimationOnce() function, this is for animation like falling, dying and reviving

Player states

The player's state is changed depending on the action, if the player is still it is set to the "Idle" animation, if moving then the state is set to "Running", etc. When the state is changed, the setFrames() function is called which looks at what state is currently set and then passes the correct frames into the m_playerFrames vector.

Level Editing

Overview

The level editor shows the grid of tiles with the outlines to easily discern and plays tiles.

Different tiles

The player can select tiles using the numbers 1-3, 1 being grass tiles, 2 being brick tiles and 3 being spikes. Then when the desired tile is selected it can be placed on the grid using left mouse button or cleared using the right mouse button.

Saving/Loading

Once the level is created, it can be saved by pressing the "S" key, this saves the level as a text file called newLevel.txt. Then for the game itself it loads files in order called level(x).txt, where x is the number of the current level. This lets me easily create levels and load them in the future.

Auto-tiling

Auto-tiling is what happens when a tile is placed and the surrounding tiles are changed to match the placed tile. As opposed to having a separate tile for each individual sprite, like the corners or edges for grass tiles, it is much easier with a function that selects the correct tile out of the tile set based on a number of if statements that check all the surrounding tiles.

References

Web-sites

Wenyao L.; Jinhua C.; Zemin L.; Rui F.; Tong H.; Lu D. (2025, March). Automatic tile position and orientation detection combining deep-learning and rule-based computer vision algorithms [Online]. (URL https://www.sciencedirect.com/science/article/abs/pii/S092658052500041X).

Yi-Si X.; Xiaoping P. L.; Shao-Ping X. (2010, June). Efficient collision detection based on AABB trees and sort algorithm [Online]. (URL https://ieeexplore.ieee.org/document/5524093).